

Roadmap de Implementación — Tienda Arduino Backend (NestJS + Prisma)

Este lienzo es tu **referente práctico** para programar **en el orden correcto**, sin perderte, y moverte más fluido (como “modo fábrica”).

Objetivo: poder construir una **V1 funcional** rápido, y que luego sea fácil iterar a V2/V3.

1) Principio clave: el orden lo dicta el flujo de negocio

En un sistema administrativo, casi siempre el orden “natural” es:

1. **Seguridad y base del proyecto** (arranque, config, errores)
2. **Catálogo** (qué vendes)
3. **Personas/actores** (clientes/proveedores/usuarios)
4. **Inventario** (stock real)
5. **Transacciones** (compras, ventas, pedidos)
6. **Extras de operación** (envíos, devoluciones)
7. **Reportes** (resumen/decisiones)

Esto evita el clásico bug mental: intentar vender cuando todavía no existe SKU, ni stock.

2) Orden recomendado por carpetas y módulos (V1)

Etapas 0 — Base mínima que siempre se hace primero

Meta: que el backend levante, responda y tenga base consistente.

Carpetas/módulos: - `src/config/` → variables de entorno, configuración central - `src/prisma/`
→ PrismaService - `src/common/` → filtros, pipes, guards, utils, logger - `src/health/` → endpoint de salud - `src/app.module.ts` / `src/main.ts`

Checklist: - `.env` funciona - migraciones ok - el servidor inicia - errores se ven bonitos (filter)

Etapas 1 — Seguridad y control de acceso

Meta: que haya login y roles antes de “tocar la data”.

Carpetas/módulos: - `src/auth/` - `src/roles/` - `src/users/`

Resultado V1: - `POST /auth/login` - `GET /auth/me` - Guard JWT funciona - Roles listos (ADMIN / VENDEDOR / BODEGA)

Etapa 2 — Catálogo (el núcleo del negocio)

Meta: estructura de productos sólida.

Carpetas/módulos: - `src/catalog/categories` - `src/catalog/products` - `src/catalog/skus`

Por qué va aquí: - Inventario, compras y ventas dependen de **SKU**

Etapa 3 — Personas (clientes y proveedores)

Meta: poder asociar transacciones a alguien.

Carpetas/módulos: - `src/customers/` - `src/suppliers/`

Etapa 4 — Inventario (stock real)

Meta: que el stock “exista como verdad” antes de vender.

Carpetas/módulos: - `src/inventory/`

Regla mental: - Stock = sumatoria de movimientos (entradas/salidas/ajustes)

Etapa 5 — Compras (entrada de stock)

Meta: que el negocio pueda reabastecer.

Carpetas/módulos: - `src/purchases/`

Regla típica: - Compra **RECIBIDA** genera movimientos de inventario positivos

Etapa 6 — Ventas (salida de stock + pagos)

Meta: la tienda gana dinero.

Carpetas/módulos: - `src/sales/`

Regla típica: - Venta **CERRADA** genera movimientos negativos de inventario

Etapa 7 — Pedidos (reservas / ventas futuras)

Meta: manejar pedidos WhatsApp (confirmado/empacado/enviado/entregado).

Carpetas/módulos: - `src/orders/` - `src/shipping/`

Por qué después de sales: - pedidos suelen ser “ventas aún no cerradas” (dependen de catálogo + clientes + reglas)

Etapa 8 — Devoluciones

Meta: operación realista del negocio.

Carpetas/módulos: - `src/returns/`

Regla típica: - una devolución genera movimiento de inventario (positivo) o ajuste

Etapa 9 — Reportes

Meta: convertir datos en decisiones.

Carpetas/módulos: - `src/reports/`

3) “Loop de productividad”: cómo programar sin bloquearte

Tu loop recomendado por cada carpeta:

1. **DTOs** (entradas)
2. **Service** (reglas)
3. **Controller** (rutas)
4. **Pruebas rápidas** (solo al final de cada carpeta)

Razón: es el flujo natural de Nest.

4) Estrategia realista de V1 → V2 → V3

V1 (suele ser 1-3 semanas)

✓ CRUD + flujo principal - login - catálogo - clientes/proveedores - compras/ventas - stock funcional - reportes básicos

No pulir UI, no fancy features.

V2 (1-3 meses)

✓ Robustez - validaciones más duras - auditoría (quién hizo qué) - performance en consultas - manejo de devoluciones más completo - reglas de negocio más finas

V3 (3-12 meses)

✓ Nivel "producto serio" - multi-sucursal - multi-caja - facturación electrónica (si aplica) - dashboards - integraciones (WhatsApp/Telegram/Email)

5) ¿Hasta qué "VX" se llega en la vida real?

En negocio real:

- V1 casi siempre se logra ✓
- V2 se logra si el cliente usa el sistema y pide mejoras ✓
- V3 depende de que el negocio crezca o cambien procesos ✓
- V4+ suele ser cuando el sistema ya es "core" del negocio

Muchas empresas se quedan en V2 felizmente.

6) Cómo usar n8n + IA para generar una V1 esqueleto (modo turbo)

La idea no es que la IA "haga todo perfecto", sino que:

✓ te genere la **estructura**, te sugiera el código repetitivo y tú revisas.

Flujo n8n recomendado (simple)

Trigger: Manual o botón

Nodo 1: Input (JSON) - nombre del módulo (ej: `inventory`) - entidades (ej: `InventoryMovement`) - endpoints deseados

Nodo 2: LLM (ChatGPT) Prompt tipo: - "Genera DTOs + service + controller para NestJS con Prisma" - "Respetar naming + estilo"

Nodo 3: Validación automática (opcional) - revisar que exporte clases - revisar nombres de rutas

Nodo 4: Output - generar archivos `.ts` como texto

Nodo 5 (opcional): Git commit draft - el flujo te devuelve una lista de archivos listos

7) Cómo preparar un generador V2/V3 (esto sí es pro)

Sí se puede, pero no como "IA mágica", sino como **plantillas**.

Idea: Generador por capas

- Capa 1: define el dominio (SKU, Sale, Purchase)
- Capa 2: define endpoints (CRUD + acciones)
- Capa 3: define reglas (transiciones de estado)
- Capa 4: genera archivos Nest

Herramientas para eso

- Nest CLI schematics (básico)
- Plop.js (templates)
- Hygen (templates)
- n8n para orquestar

En la vida real, tu generador te ahorra 40–60% del trabajo repetitivo.

8) La regla más importante para que no se te muera el proyecto

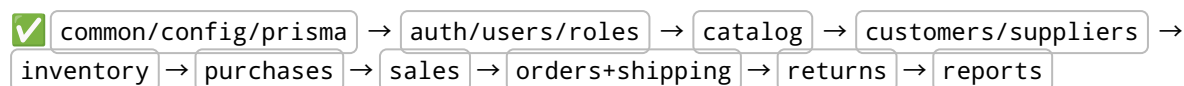
Nunca generes todo en una sola tanda.

Hazlo por “módulos” y cada módulo cumple:

- compila ☒
- endpoints existen ☒
- reglas mínimas ☒

Luego recién pasas al siguiente.

9) Tu orden rápido para ser más fluido (resumen 1 línea)



10) Cómo usar este lienzo en práctica

Cuando vayas a programar, pregúntate:

- “¿Esto depende de qué?”
- “¿Ya existe el ID que necesito?”
- “¿Este módulo impacta stock o dinero?”

Si impacta stock o dinero, se prueba con más cuidado.

Si quieres, el siguiente lienzo lo armamos como: ☒ **Checklist de endpoints V1** (todos los) por orden, para que tu implementación sea casi mecánica.